Group Project

Phoenix, Munem, and Joel

Vehicle Maintenance Tracking System

Repo: https://github.com/HoelJ/Vehicle-Maintenance-Tracking-System
Trello: https://trello.com/b/FZjoDZfE/cis344groupproject

Project plan

Objective: This website will provide workers with simple forms that can be used to input information into a database that keeps track of vehicles, customers, services, employees, and billing information.
Scope: This website will be limited to a home page that provides links to the forms. The forms allow the user to input valid information, and will either display a completed screen, or an error screen, depending on if there is an issue with the posting of the data.
Design Plans: This website will have a home page with buttons. The buttons will provide links to the individual forms for services, customers, and vehicles.

Technical Details

HTML: Will be used to provide the text and form of all pages on the website.

Code example snippet:

```
<head>
<h1 style="text-align: center; font-size: 50px">Vehicle Maintenance Tracking System</h1>
</head>
<h2 style="text-align: center; font-size: 35px">What would you like to do?</h2>
<table style="text-align: center; font-size: 20px">
 <tr>
  <td>
  <h2>Vehicles</h2>
  </td>
  <td>
  <h2>Services</h2>
  </td>
  <td>
   <h2>Customers</h2>
  </td>
 </tr>
 <tr>
  <td>
  <a href="new-vehicle-form.php" class="button">Go to Vehicle Form</a>
  </td>
  <td>
  <a href="new-service-form.php" class="button">Go to Service Form</a>
  </td>
  <td>
  <a href="new-customer-form.php" class="button">Go to Customer Form</a>
  </td>
```

In this example, the <table> tag is used to create a table with columns that will show form choices in the first row, and links to those forms in the second row.

CSS: Will be used to create aspects such as buttons, as well as to modify appearance (font size, bold, etc,.)

```
.button {
  background-color: #067aff;
  color: #fffbf2;
  display: inline-block;
  width: 200px;
  length: 200px
  font-size: 16px;
  cursor: pointer; /* Change mouse cursor on hover */
  transition: background-color 0.3s;
  border-radius: 16px;
  text-decoration: none;
  padding: 20px;
  font-weight: 500; /*Slightly bold text*/
}
```

In this example, we define a class called button, which we will apply to the elements in the table, so that they can look like buttons that when pressed, send you to the form.

SQL: Will be used to create the database, and in conjunction with PHP to insert data into tables.
Code snippet example:

```
/* Create tables and initialize datatypes/ constraints */
CREATE TABLE Customers (
  customerID INT NOT NULL PRIMARY KEY,
  Name VARCHAR(45) NULL,
  dateJoined DATE NULL,
  Age INT NULL,
  Email VARCHAR(45) NULL,
  Phone VARCHAR(14) NULL

  );
```

In this example, SQL creates for the entity customers, as well as its attributes: customerID, Name, dateJoined, Age, Email, and Phone. This gives us a place to store all relevant data about a customer.

PHP: Will be used to move the information gathered in the HTML forms into the database. The data from the HTML post will be put into variables, processed, and sent to the database.
Code example snippet:

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $vin = $_POST['VIN'];
    $vehicle_type = $_POST['vehicle_type'];
    $model = $_POST['model'];
    $make = $_POST['make'];
    $year = $_POST['year'];
    $cust_ID = $_POST['cust_ID'];
```

In this example, the data input into the form is placed into PHP variables so that they can later be validated and sent to the SQL database.

Development Process:
Phoenix designed and created the SQL database. Joel and Munem worked together to develop the front-end using HTML and CSS, while integrating PHP to communicate with the SQL server. Then, all three members worked together to fix any errors that arose regarding the connection between the site and the database.

Database Design

PHOENIX

For the design of the database, I took the approach of designing the way the data is stored to be of use for someone who wants to manage customer records, the vehicles they own as well as the services that have been done to them. A few other tables are also included with holding information about mechanics who work on vehicles, as well as details about service transactions and billing to keep track of balances for different services.
Some things we need to keep track of:

- VIN (identification number)
- Year
- Model
- Color
- Last Maintenance
- Maintenance ID
- Maintenance types

What is the purpose of this database?:
As mentioned, this database will aid in tracking service history for vehicles, retaining information about customers who own the vehicles and the details about each service.
Key features:
Joins for vehicles, service records, and mechanics; constraints for maintenance intervals; transactions for service updates; secure data management.

Schema:
Entities:

- Vehicles (vehicleID(PK), CustomerID(FK) vehicleType, Model, Make, year)
- Customers(customerID(PK), name, dateJoined, age, email, phone)
- Mechanics(mechanicID(PK), name, specialization, hourlyRate, phone)
- Service(serviceID(PK), vehicleID(FK), serviceDate, typeOfService, status, totalCost)
- ServiceDetails(serviceDetailID(PK), serviceID(FK), mechanicID(FK), description(opt.), LaborCost, PartsUsed)
- Billing(BillID(PK), ServiceID(FK), billingDate, amtPaid, paymentMethod)

Relationships:
Mechanics | serviceDetails (1:N)
serviceDetails | Service (N:1)
Service | Billing (1:N)
Service | Vehicles (N:1)
Vehicles | Customers (N:1)
Joins:

```
SELECT Model, Make

FROM Vehicles

NATURAL JOIN Customers;
```

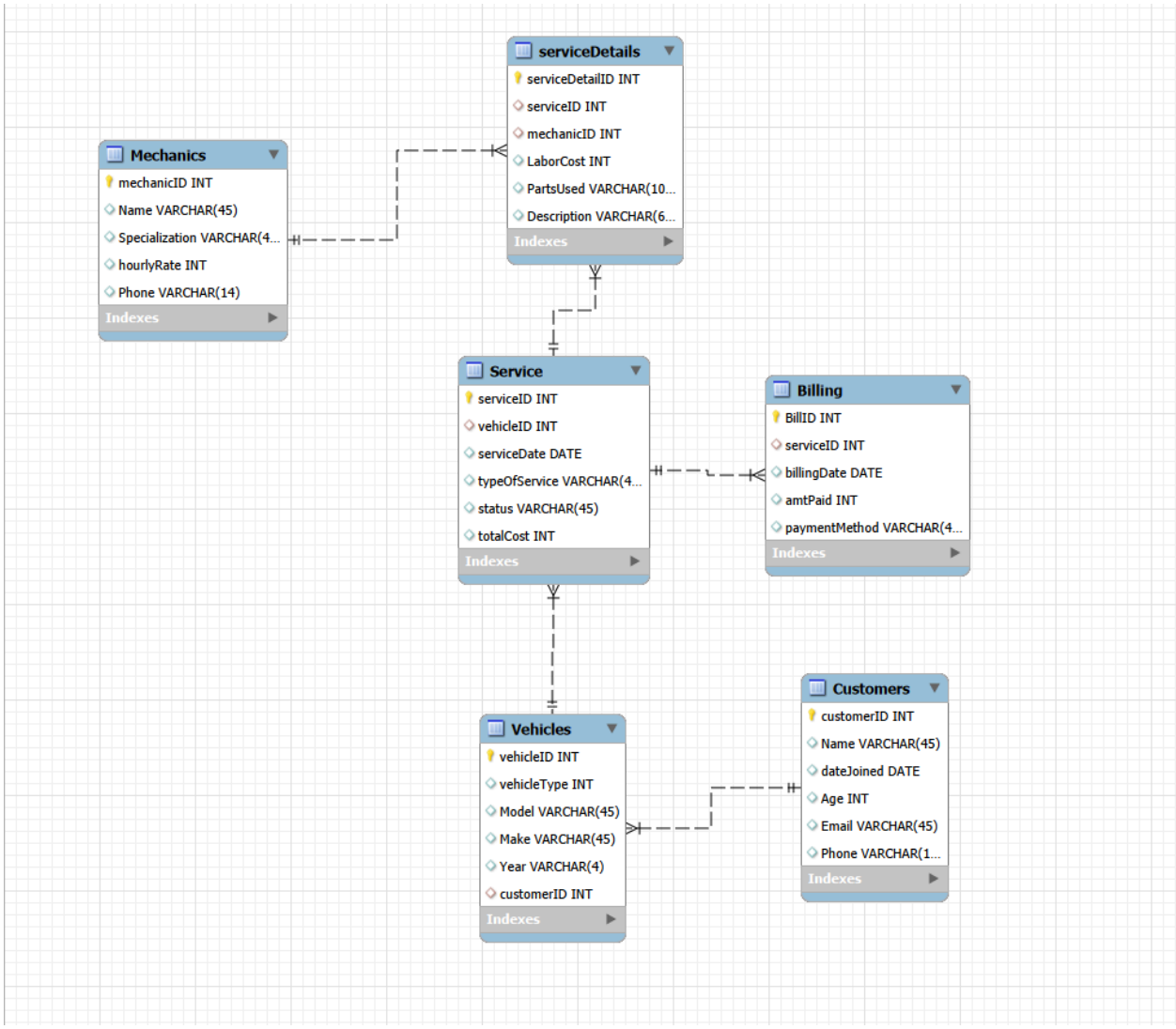
```
SELECT Mechanics.mechanicID, Mechanics.Name,
serviceDetails.serviceDetailID, serviceDetails.PartsUsed

FROM Mechanics

LEFT JOIN serviceDetails

ON Mechanics.mechanicID = serviceDetails.mechanicID;
```

```
SELECT Mechanics.Name, Mechanics.mechanicID, serviceDetails.PartsUsed

FROM Mechanics

RIGHT JOIN serviceDetails

ON Mechanics.mechanicID = serviceDetails.mechanicID;
```

```
SELECT Service.serviceID, Service.typeOfService, Vehicles.vehicleType
FROM Vehicles
JOIN Service
ON Vehicles.vehicleID = Service.vehicleID;
```

```
SELECT Billing.BillID, Billing.billingDate, Service.serviceDate,
Service.totalCost
FROM Billing
FULL JOIN Service
ON Billing.serviceID = Service.serviceID;
```

ERD:



All SQL queries and joins will be in the .mysql script, including the forwarder code for creating the database from the schema.

PHP and MySQLI: I was able to use the MySQLI extension in order to establish a connection with the database, then use that connection to either insert (create anew) or update an existing record.
In the database.php file,

Vehicle-Maintenance-System > database.php

```php
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "vehicle_service_db";
try{
$conn = mysqli_connect($servername, $username, $password, $dbname);
}
catch(mysqli_sql_exception){
    echo"Could not connect...";
}

if($conn){
    echo"Connected.";
    echo "<br>";
}

?>
```

We establish the connection and use the try block to catch any exceptions that may arise and cause no connection.
Then, in each record (Ex. new-customer-record.php)

```php
$sql = "UPDATE customers SET Name=?, dateJoined=?, Age=?, Email=?, Phone=? WHERE customerID=?";
try{
    $stmt= $conn->prepare($sql);
    $stmt->bind_param("ssissi", $cust_name, $date_joined, $cust_age, $cust_email, $cust_phone, $cust_ID);
    if ($stmt->execute()) {
        echo "<h3 style='text-align:center; color:green; font-size:25px'>Customer record updated successfully!</h3>";
        echo '<a href="Home_Page.php">Go back</a>';
    }
} catch(mysqli_sql_exception $exception) {
        echo "<h3 style='text-align:center; color:red; font-size:25px'>Error: " . $exception->getMessage(); "</h3>";
        echo "<br>";
        echo "<a href='Home_Page.php'>Go back</a>";
```

```php
$sql = "INSERT INTO customers (customerID, Name, dateJoined, Age, Email, Phone)
        VALUES ('$cust_ID','$cust_name','$date_joined','$cust_age','$cust_email','$cust_phone');";
    try {
    $result = mysqli_query($conn, $sql);
        if ($result) {
            echo "<h3 style='text-align:center; color:green; font-size:25px'>Customer added successfully!</h3>";
            echo '<a href="Home_Page.php">Go back</a>';
        }
        } catch(mysqli_sql_exception $exception) {
            echo "<h3 style='text-align:center; color:red; font-size:25px'>Error: " . $exception->getMessage(); "</h3>";
            echo "<br>";
            echo "<a href='Home_Page.php'>Go back</a>";
        }
```

We either Update or insert a new row using branching with If-else statements. For updating, we use $stmt to prepare the query to ensure no SQL injection, use bind_param to format how the data will be expected and execute the query. From there, we check if the query actually executed, or use catch to handle any exceptions.
For inserting, we just run the query and store the result in a variable called $result, and handle exceptions just like when updating.

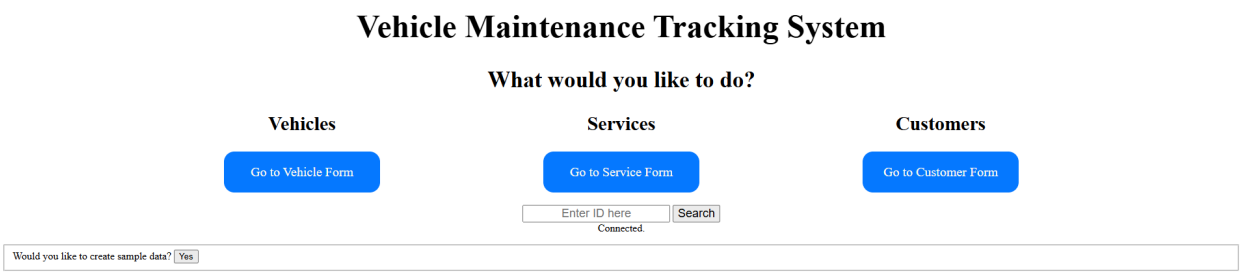
User tutorial:

First, ensure you have PHP set up with a XAMPP server locally hosting the site on your device.

Then, download all of the files from the github, and keep them saved in a folder of the same name as the original github. Save this folder inside of the htdocs folder for the xampp web server files.

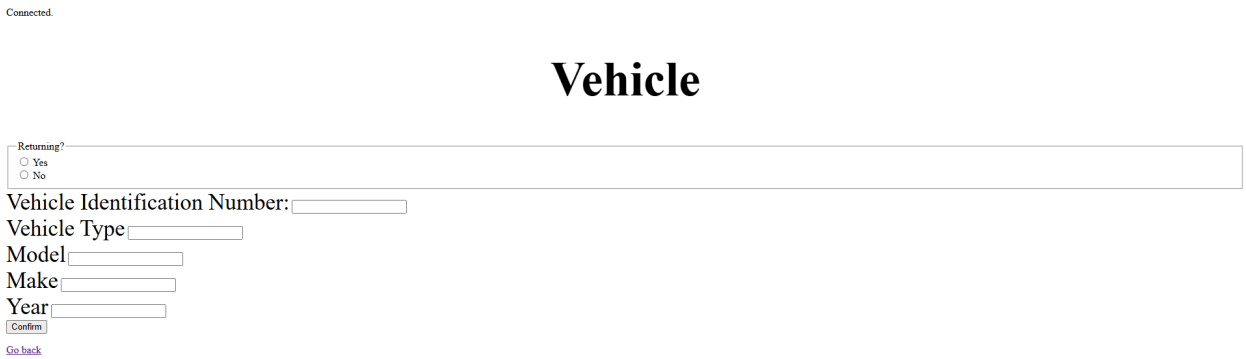
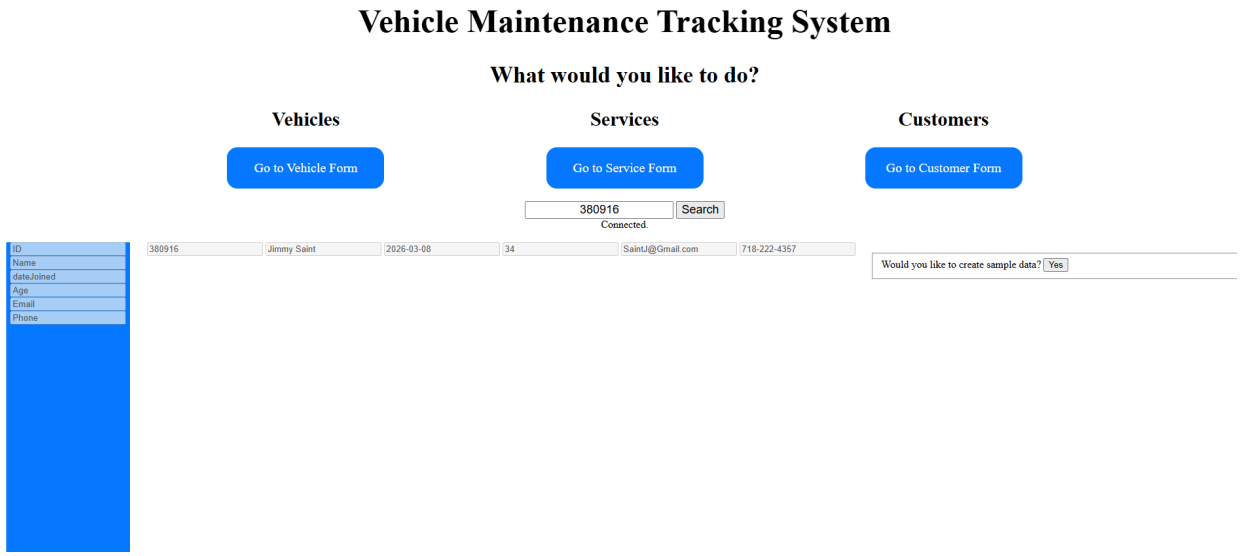
Next, use the Xampp control panel application to open up the admin page for the MySQL service (PHP MyAdmin). Here is where you will take the SQL file from the github and paste the queries up until the join statements to create, and initialize all of the tables in the database for handling the data. All the relationships, foreign keys and constraints are all already set up to be used immediately.

From here, you can open the website under the localhost on your web browser, where you will be greeted with this screen:



Here is where you will be able to do a bunch of things. If you have data to sample with this already, perfect. If not, there is a sample button on the homepage which will automatically fill the tables with data used for testing the site's functionality.

In each section (vehicles, customers, service), the user has the ability to either create a new form, or update an existing one using the “returning?” radio button selection, which the program will use to branch to either an INSERT or UPDATE SQL query, achieved through MySQLI (as shown in the code)



In addition, the homepage incorporates a search feature which allows for a user to search for all records of a specific customerID. The results are displayed below it and show all matching results using a SELECT query.